

1. Abstracción y Polimorfismo

La Programación Orientada a Objetos (POO) no es solo una técnica de codificación, sino un paradigma para analizar, diseñar y construir sistemas de software que afronten la complejidad de los requisitos modernos. Dos de sus bases fundamentales son la **abstracción** y el **polimorfismo**, las cuales permiten crear modelos que se asemejan al pensamiento humano. La abstracción y el polimorfismo son los “**Pilares de la Flexibilidad**”.

El Concepto de Abstracción

La **abstracción** se define como un proceso mental de extracción de las características esenciales de un objeto, ignorando deliberadamente los detalles superfluos o irrelevantes para un contexto dado. Es un proceso subjetivo que depende del interés del observador; por ejemplo, una persona puede ser abstraída como un "Empleado" en un sistema contable o (la misma persona) como un "Paciente" en un sistema médico.

En Java, la abstracción permite concentrarse en **qué hace** un objeto en lugar de **cómo lo hace**. Este concepto se materializa mediante el uso de clases y métodos que definen comportamientos generales que luego serán concretados por entidades más específicas.

El Polimorfismo y la "Multiplicidad de Formas"

El término **polimorfismo** proviene de las raíces *poli* (muchas) y *morfismo* (formas). En el paradigma de objetos, se refiere a la capacidad de una variable de referencia para adoptar múltiples formas, permitiendo que objetos de diferentes tipos respondan al mismo mensaje de maneras distintas.

El uso más común del polimorfismo ocurre cuando se utiliza una referencia de una **clase madre** (superclase) para referirse a un objeto de una **clase hija** (subclase). Esto permite tratar a una colección de objetos diversos de forma genérica, siempre que compartan una raíz común en la jerarquía de herencia.

2. Profundizando en el Polimorfismo

El polimorfismo se manifiesta de diversas maneras en Java, principalmente a través de la relación entre la signatura (o firma) de los métodos y el momento en que el lenguaje decide qué código ejecutar.

Sobrescritura versus Sobrecarga

Es vital distinguir entre estos dos conceptos, ya que representan formas distintas de polimorfismo (estático y dinámico).

1. **Sobrecarga de Métodos (Overloading):** Ocurre dentro de una **misma clase**. Permite tener múltiples métodos con el mismo nombre pero con **diferente signatura** (distinta cantidad, tipo u orden de parámetros). Es considerada una forma de polimorfismo estático porque el compilador resuelve qué método invocar en tiempo de compilación basándose en los argumentos pasados.
 - *Ejemplo:* Un método saludar () sin parámetros y un método saludar(String nombre) con un parámetro.
2. **Sobrescritura de Métodos (Overriding):** Ocurre en el contexto de la **herencia**. Una subclase redefine un método que ya existe en su superclase, manteniendo la **misma signatura** (nombre y parámetros) pero cambiando su comportamiento interno. Se utiliza la anotación `@Override` para indicar explícitamente esta intención y evitar errores involuntarios.

Enlace Dinámico (Dynamic Binding)

El **enlace dinámico** es el mecanismo que permite al polimorfismo actuar en tiempo de ejecución. Cuando se invoca un método sobre una variable de referencia, Java no mira el tipo de la variable, sino el **tipo real del objeto** almacenado en ella para decidir qué implementación ejecutar.

Este proceso permite que el código sea altamente extensible. Si tenemos una lista de Animales y llamamos al método hacerSonido(), el enlace dinámico asegurará que el Perro ladre y el Gato maúlle, sin que el programador deba usar condicionales if-else o switch para distinguir los tipos.

El Principio de Sustitución y la Regla "Es un"

Para que el polimorfismo sea coherente, debe basarse en el **principio de sustitución**: un objeto de una subclase debe poder reemplazar a un objeto de su superclase sin romper la lógica del programa. Esto se valida con la regla semántica **"Es un"**: por ejemplo, un Director "es un" Empleado, por lo tanto, puede ser tratado como tal.

3. Clases y Métodos Abstractos

En el diseño de jerarquías, a menudo nos encontramos con clases que son demasiado generales para ser instanciadas.

Definición y Propósito

Una **clase abstracta** es aquella que no permite la creación de instancias (objetos) de sí misma. Se utiliza como un "esqueleto" o plantilla para que otras clases hereden de ella y completen su funcionalidad. Se definen anteponiendo la palabra reservada `abstract` en la declaración de la clase.

Métodos Abstractos: Contratos de Implementación

Un **método abstracto** es una declaración de método que no tiene implementación (no tiene cuerpo, solo la firma/signatura terminada en punto y coma). Su función es obligar a las subclases concretas a proporcionar su propia implementación de dicho método.

- **Consecuencia fundamental:** Si una clase contiene al menos un método abstracto, la clase completa debe ser declarada como abstracta.

Diseño de Jerarquías Flexibles

El uso de clases abstractas permite diseñar jerarquías robustas donde el comportamiento común se centraliza en la superclase (para evitar duplicación de código mediante el principio DRY que se verá mas adelante) y el comportamiento específico se delega a las subclases.

Ejemplo de diseño: En una jerarquía de `FiguraGeométrica`, la clase base puede tener un método abstracto llamado `perímetro()`. Cada figura concreta (`Círculo`, `Rectángulo`) sabrá cómo calcularlo, pero la clase base garantiza que **todas** las figuras tendrán obligatoriamente esa capacidad.

4. Interfaces: Definición de Contratos

Cuando necesitamos que objetos de diferentes jerarquías compartan comportamientos, o cuando queremos un nivel de abstracción aún más puro, recurrimos a las **interfaces**.

Definición de Contratos Puros

Una **interfaz** es un tipo abstracto que representa un conjunto de métodos que una clase se compromete a implementar. A diferencia de las clases abstractas, las interfaces tradicionalmente solo contenían firmas de métodos (totalmente abstractos) y constantes estáticas finales.

En Java, se definen con la palabra `interface` y se implementan con `implements`. Representan un "contrato" legal: si una clase dice que implementa una interfaz, debe

proveer código para todos sus métodos, o de lo contrario, la clase debe declararse abstracta.

Implementación Múltiple

Java no permite la herencia múltiple de clases para evitar ambigüedades, pero sí permite que una clase implemente **múltiples interfaces**. Esto permite que un objeto tenga múltiples "roles". Por ejemplo, una clase Ornitorrinco puede heredar de Mamífero e implementar también la interfaz Ovíparo.

Diferencias Clave

- **Clase Abstracta:** Representa lo que un objeto **es** (similar a la herencia simple). Puede tener estado (atributos) y código compartido (métodos).
- **Interfaz:** Representa lo que un objeto **puede hacer**. Es una declaración de capacidades puras (normalmente mediante métodos).

5. Principios Básicos de Diseño OO

El buen diseño no se trata solo de usar la sintaxis correcta, sino de seguir principios que garanticen la calidad del software a largo plazo.

Alta Cohesión y Bajo Acoplamiento

Estos son los dos pilares de la modularidad.

1. **Cohesión:** Mide el grado de conectividad entre los elementos internos de un módulo o clase. Buscamos una **alta cohesión**, donde una clase se enfoque en una sola tarea clara y todos sus métodos estén relacionados con esa tarea.
2. **Acoplamiento:** Mide el nivel de dependencia entre diferentes módulos o clases. Buscamos un **bajo acoplamiento**, minimizando las interdependencias para que los cambios en una clase no obliguen a reconstruir o modificar muchas otras clases. El polimorfismo es la herramienta principal para reducir el acoplamiento.

Principio de Responsabilidad Única (SRP)

Derivado del acrónimo SOLID, establece que una función o clase debe tener **una sola responsabilidad** y, por tanto, una única razón para cambiar. Si una clase o función hace demasiadas cosas, se vuelve difícil de entender y mantener.

6. Patrones Simples: Desacoplamiento

Los patrones de diseño son soluciones probadas a problemas recurrentes en el diseño de software.

El Patrón Strategy (Estrategia)

El patrón **Strategy** permite definir una familia de algoritmos, encapsular cada uno de ellos y hacerlos intercambiables. En lugar de implementar un comportamiento directamente en una clase, se delega a un objeto "estrategia" a través de una interfaz. Esto permite cambiar el comportamiento de un objeto en tiempo de ejecución simplemente inyectándole una implementación diferente de la interfaz.

Uso de Interfaces para Desacoplar

Las interfaces son la herramienta por excelencia para desacoplar el código. Cuando una clase A depende de una interfaz I en lugar de una clase concreta B, hemos roto la dependencia directa. Esto permite que el sistema sea **escalable**: podemos añadir nuevas implementaciones de I sin tocar el código de la clase A.

7. Buenas Prácticas de Programación

Programar bien consiste en construir soluciones que otros (incluido nuestro "yo" del futuro) puedan entender y mejorar.

Nombres, Organización y Legibilidad

- **Nombres que expresen intención:** Las variables, métodos y clases deben indicar claramente qué representan o hacen sin necesidad de comentarios explicativos.
 - **Clases/Interfaces:** Sustantivos, empezando con mayúscula (PascalCase).
 - **Métodos:** Verbos, empezando con minúscula (camelCase).
 - **Variables:** Sustantivos en camelCase.
 - **Constantes:** Mayúsculas, separando palabras con guiones bajos (_).
- **KISS (Keep It Simple, Stupid):** La solución más simple suele ser la mejor. Se debe evitar el código excesivamente compacto o complejo si una versión más clara o sencilla es posible.

Reutilización y Extensibilidad

- **DRY (Don't Repeat Yourself):** Cada pieza de lógica debe tener una representación única en el código. La duplicación de código dificulta el mantenimiento y propicia errores inconsistentes.
- **Encapsulamiento:** Los campos de una clase deben ser **privados**. El acceso se debe realizar a través de métodos públicos (getters y setters), lo que permite validar los datos y proteger el estado interno del objeto.
- **Inmutabilidad:** Siempre que sea posible, se deben preferir objetos cuyos campos no puedan cambiar tras la construcción, lo que garantiza un estado siempre válido.

Resumen de Conceptos Clave

Concepto	Definición	Cuando se aplica
Abstracción	Capturar lo esencial, ignorar lo superfluo.	Diseño
Clase Abstracta	Molde general, no instanciable.	Diseño
Interfaz	Contrato de comportamiento puro.	Diseño
Sobrecarga	Mismo nombre, distintos parámetros.	Compilación
Polimorfismo	Una interfaz, múltiples implementaciones.	Ejecución
Sobrescritura	Mismo nombre y parámetros, distinta lógica.	Ejecución
Enlace Dinámico	Decisión del método según el tipo real del objeto.	Ejecución

Se han resumido los conocimientos técnicos necesarios para diseñar sistemas robustos y flexibles bajo el paradigma Orientado a Objetos, enfatizando que el diseño correcto (abstracción e interfaces) es tan importante como la implementación técnica (polimorfismo y herencia).